LAB NO: 9
MEMORY MANAGEMENT I

1. Write a C/C++ program to simulate First-fit, Best-fit and Worst-fit strategies. Given memory partitions of 100K, 500K, 200K, 300K, and 600K(in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)?

Program:

```
#include <stdio.h>

void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d (%d) -> ", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("Block %d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nBest Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d (%d) -> ", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("Block %d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    printf("\nWorst Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d (%d) -> ", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("Block %d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
```

```
    int blockSize[m], b1[m], b2[m];

    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);
    sort(blockSize, m);

    for (int i = 0; i < m; i++) {
        b1[i] = blockSize[i];
        b2[i] = blockSize[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];
    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    firstFit(blockSize, m, processSize, n);
    bestFit(b1, m, processSize, n);
    worstFit(b2, m, processSize, n);

    return 0;
}
```

Output:

```
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100000
500000
200000
300000
600000
Enter number of processes: 4
Enter sizes of processes:
212000
417000
112000
426000

First Fit Allocation:
Process No.     Process Size    Block No.
1               212000          2
2               417000          5
3               112000          2
4               426000          Not Allocated

Best Fit Allocation:
Process No.     Process Size    Block No.
1               212000          4
2               417000          2
3               112000          3
4               426000          5

Worst Fit Allocation:
Process No.     Process Size    Block No.
1               212000          5
2               417000          2
3               112000          5
4               426000          Not Allocated
```

2. Assuming a page size of 32 bytes and there are total of 8 such pages totaling 256 bytes. Write a C program to simulate this memory mapping. The program should read the logical memory address and display the page number and page offset in decimal. How many bytes do you need to represent the address in this scenario? Display the page number and offset to reference the following logical addresses.
(i) 204 byte
(ii) 56 byte

Program:

```
#include <stdio.h>

int main()
{
   int page_size = 32;
   int total_memory = 256;
   int logical_address;

   printf("Enter logical address: ");
   scanf("%d", &logical_address);

   int page_number = logical_address / page_size;
   int offset = logical_address % page_size;

   printf("\nPage Number = %d\n", page_number);
   printf("Offset = %d\n", offset);

   int temp = total_memory;
   int bits = 0;

   while (temp > 1)
   {
      temp = temp / 2;
      bits++;
   }

   printf("Number of bits required = %d\n", bits);

   return 0;
}
```

Output: